Dynamic Timing-Perturbation Validation for Potential iOS Kernel MTE Allocator-State Races

Kaya Ercihan

03.08.2026

Abstract

This report is the third dynamic-testing addition to the iOS kernel MTE allocator-state investigation on a physical **iPhone 17e / iOS 26.5.1**. It documents only the implementation and experiments not covered by the preceding four reports: user-configurable NOP timing jitter, runtime-generated `MAP_JIT` sleds, the signed static-sled fallback, mixed ARM64 timing instructions, and a fifteen-execution comparison across five zone-pressure modes. The five modes were executed under three configuration families: baseline, the *Realtime priority (FLOOD)* option, and NOP jitter with a requested 64-entry sled pool using the *Mixed NOP-equivalents* setting. The new implementation inserts a selected timing sled immediately before controlled Mach allocation or release syscalls, thereby changing relative kernel-entry timing without changing the target syscall sequence. Source review establishes two interpretation constraints. First, the real-time request is invoked only by FLOOD-family workers; enabling the switch for FAST or PIPELINE does not call the real-time policy path. Second, mixed instruction emission is available only when a `MAP_JIT` backend succeeds; the static fallback contains pure NOP sleds. All fifteen executions were negative for an externally generated MTE tag-fault signature under the stated Console filters. The result increases black-box configuration coverage, but it does not prove that the helper was skipped, that mixed JIT code executed in every requested run, or that the same-object instruction-level race is absent.

1 Scope and implementation continuity

The static helper analysis, userland-to-XNU path correlation, syscall timing limitation, allocator-pressure modes, Skywalk serialization assessment, background wrapper, and earlier run counters are documented in the four preceding repository reports [1, 2, 3, 4]. This report is a compact implementation-and-test delta: it adds the NOP/JIT timing layer and the fifteen-execution matrix without reprinting every earlier listing.

1.1 Continuity across the five-paper series

The current source revision contains the userland and dynamic methods introduced across Reports 01–04. Report 00 is the static reverse-engineering basis and therefore has no userland feature that must be "implemented." Table 1 makes that boundary explicit.

Table 1: Five-paper continuity and concrete source anchors

Report	Contribution	Status in the inspected source revision
00 Static analysis	Helper/caller disassembly, MTE stores, and ordering hypothesis.	Analysis basis only; no stock-userland implementation is expected.
01 Trigger-path correlation	Mach IPC/zone and BSD/Skywalk trigger surfaces.	Implemented by `mte_trigger_zone_path()` and `mte_trigger_skywalk_path()`.
02 Reachability and timing limits	Zone-only parallel pressure, cross-thread allocation/free separation, affinity-guided CPU placement, overlap counters, and Skywalk exclusion.	Implemented by `mte_trigger_parallel()`, affinity tags, SPSC handoff rings, and atomic counters.
03 Allocator-pressure validation	FAST, PIPELINE, FLOOD, `PREFILL_FLOOD`, `PENDING_FLOOD`, real-time control, background execution, and reports.	Implemented in `mte_trigger.c/.h` plus `MTETriggerApp.swift` and `TriggerViewModel.swift`.
04 This report	Loop jitter, `MAP_JIT` sleds, static signed fallback, mixed instruction emission, and fifteen-run fault triage.	Implemented in `jit_nop.c/.h` and wired into every applicable zone worker path.

The evidence model separates source-code findings from run evidence. Source review establishes what each mode and backend can execute. The author-supplied matrix identifies the requested configurations and reports

that all fifteen executions completed without a visible tag-fault symptom. The available Console capture verifies that the only `DFSC 0x11` matches were emitted by `MTETrigger.app` itself; it does not preserve complete per-run counters, JIT backend lines, or real-time grant results [5].

2 New timing-perturbation implementation

2.1 Configuration path from SwiftUI to the C workers

The settings view adds independent controls for the jitter ceiling, sled-pool enablement, pool size, and instruction mix. The view model forwards these values to the C backend immediately before `mte_trigger_parallel()`. In the tested configuration the requested pool size was 64 and the requested mix was `JIT_NOP_MIXED` [6, 7].

```
let nops: UInt32 = nopEnabled ? nopMax : 0
let jitEnabled = jitNopEnabled
let jitPool = jitPoolSize
let jitMix = jitMixMode

mte_set_nop_max(nops)
mte_set_jit_nop_enabled(jitEnabled)
mte_set_jit_pool_size(jitPool)
mte_set_jit_mix_mode(jitMix)
mte_set_realtime_enabled(rtEnabled)
mte_trigger_parallel(workers, duration, effectivePorts, mode)
```

Listing 1: Swift-to-C propagation of the new timing controls

2.2 Mode dispatch and realized worker class

The orchestrator converts the five UI modes into two concrete worker implementations. FAST and PIPELINE use batch rings; FLOOD, PREFILL_FLOOD, and PENDING_FLOOD all use the single-port FLOOD workers. The prefill variants change allocator state before those workers start.

```
bool is_prefill = (mode == MTE_MODE_PREFILL_FLOOD);
bool is_pending_flood = (mode == MTE_MODE_PENDING_FLOOD);
bool is_flood = (mode == MTE_MODE_FLOOD || is_prefill ||
    is_pending_flood);

if (is_prefill) {
    reserve_count = num_ports;
    num_ports = 1; //FLOOD hot path
} else if (is_pending_flood) {
    pending_port_count = (num_ports == 0) ? 1000 : num_ports;
    num_ports = 1; // FLOOD hot path
} else if (num_ports == 0) {
    num_ports = is_flood ? 1 : 8; // FAST/PIPELINE batch default
}

if (is_prefill && reserve_count > 0)
    prefill_zone(reserve_count, &prefill_ports, &actual_prefill);

if (is_pending_flood && pending_port_count > 0)
    pending_prefill_zone(pending_port_count, msgs_per,
        &pending_ports, &actual_pending);

// Exact worker-family dispatch -> context assignments omitted
if (is_flood) {
    pthread_create(&threads[idx], NULL,
        zone_flood_alloc_worker, &ctxs[idx]);
    idx++;
    pthread_create(&threads[idx], NULL,
        zone_flood_free_worker, &ctxs[idx]);
    idx++;
} else {
    pthread_create(&threads[idx], NULL,
        zone_alloc_worker, &ctxs[idx]);
    idx++;
    pthread_create(&threads[idx], NULL,
        zone_free_worker, &ctxs[idx]);
    idx++;
}
```

Listing 2: Mode classification, pre-run setup, and worker-family dispatch

The C layer stores the settings in atomics. The pool-size control is clamped to 4–256 entries, the UI exposes a maximum NOP count from 1 to 10000 and defaults to 100. A pool entry is a callable straight-line function ending in `RET`. Runtime selection uses `arc4random_uniform(pool_size)`, so each syscall attempt can receive a different pre-entry delay.

2.3 Three timing backends

The implementation contains three effective timing paths. The non-pool path executes a deterministic loop. The preferred pool backend emits code into `MAP_JIT` memory. If that mapping fails, pool creation falls back to signed, precompiled functions in the application text segment [8].

Table 2: Implemented timing backends

Backend	Selection and construction	Timing property
Loop NOP	Used when the pool is disabled or unavailable; executes `nop_sled(i % N)` or `nop_sled(iter % N)`.	Deterministic repeating length and loop branch/compare overhead.
`MAP_JIT` pool	Allocates 64 entries in the tested configuration; each entry receives a random length from 1 to `max_nops` and is emitted once before the run.	Random pool construction plus random entry selection at each worker iteration; pure or mixed emission is supported.
Static signed pool	Randomly selects from 29 precompiled naked functions with NOP counts from 1 to 10000, filtered by `max_nops`.	Random entry selection without writable executable memory; the emitted instructions are pure NOP only.

Pool size denotes callable entries, not necessarily unique lengths. Under the JIT backend, two or more entries can receive the same random length. Under the static backend, entries are selected with replacement from the eligible precompiled table.

2.4 Mixed instruction emission

The mixed mode is implemented as an instruction-encoding table consumed by the JIT emitter. Two entries are architectural NOPs, while the other entries are register-preserving instructions or hints. Each generated sled ends in `RET` [8, 9].

```
#define ARM64_NOP       0xD503201F
#define ARM64_YIELD     0xD503203F
#define ARM64_SEV       0xD503209F
#define ARM64_MOV_X0_X0 0xAA0003E0
#define ARM64_MOV_X1_X1 0xAA0103E1
#define ARM64_MOV_X2_X2 0xAA0203E2
#define ARM64_MOV_W0_W0 0x2A0003E0
#define ARM64_RET       0xD65F03C0

static const uint32_t nop_variants[] = {
    ARM64_NOP, ARM64_NOP,
    ARM64_YIELD, ARM64_SEV,
    ARM64_MOV_X0_X0, ARM64_MOV_X1_X1,
    ARM64_MOV_X2_X2, ARM64_MOV_W0_W0,
};

static uint32_t *emit_sled(uint32_t *buf, uint32_t count,
    uint32_t mix_mode)
{
    for (uint32_t i = 0; i < count; i++)
        buf[i] = (mix_mode == JIT_NOP_MIXED)
            ? nop_variants[arc4random_uniform(NOP_VARIANT_COUNT)]
            : ARM64_NOP;
    buf[count] = ARM64_RET;
    return &buf[count + 1];
}
```

Listing 3: Actual mixed-instruction table and sled emitter

The duplicated NOP entry weights NOP at two of eight random choices. The term *NOP-equivalent* should be read as *register-state-preserving timing perturbation*, not cycle-identical execution: `YIELD` is a scheduling hint and `SEV` interacts with the event mechanism. This is useful for timing diversity but is not a pure-NOP control.

2.5 JIT memory handling and signed fallback

The runtime path first attempts `MAP_JIT` with read/write/execute permissions, then a read/write `MAP_JIT` mapping. It resolves `pthread_jit_write_protect_np()` dynamically, disables JIT write protection while emitting, restores protection, and invalidates the instruction cache. The development entitlements request `com.apple.security.cs.allow-jit` together with `get-task-allow` and disabled library validation [8, 10].

Apple documents `com.apple.security.cs.allow-jit` as the Hardened Runtime entitlement associated with creating writable and executable memory through `MAP_JIT`. In the tested iOS development build, requesting that entitlement did not make the `MAP_JIT` backend available. Whether the fast path in Listing 4 succeeds is therefore a property of the realized runtime code-signing and debugger environment, not of the requested entitlement set alone [11].

```
base = mmap(NULL, total, PROT_READ | PROT_WRITE | PROT_EXEC,
    MAP_PRIVATE | MAP_ANON | MAP_JIT, -1, 0);
if (base == MAP_FAILED) {
    base = mmap(NULL, total, PROT_READ | PROT_WRITE,
        MAP_PRIVATE | MAP_ANON | MAP_JIT, -1, 0);
}
if (base == MAP_FAILED)
    return NULL;

jit_write_protect(false);
memset(base, 0, total);
for (uint32_t i = 0; i < pool_size; i++) {
    uint32_t n = arc4random_uniform(max_nops) + 1;
    uint32_t *start = (uint32_t *)((uint8_t *)base +
        (size_t)i * sled_stride);
    emit_sled(start, n, mix_mode);
    pool->sleds[i].code = start;
    pool->sleds[i].nop_count = n;
}
jit_write_protect(true);
sys_icache_invalidate(base, total);

// Public factory: MAP_JIT first, signed .text fallback second
pool = try_jit_backend(pool_size, max_nops, mix_mode);
if (pool) return pool;
return try_static_backend(pool_size, max_nops); // pure NOP
```

Listing 4: MAP_JIT pool construction, cache publication, and signed fallback

This fallback is reproducibility-critical. Selecting *Mixed NOP-equivalents* in the UI records a requested mix, but the executed mix is mixed only if a `MAP_JIT` backend succeeds. When strategy 5 is selected, the source explicitly reports *pure (static)*. Consequently the following runtime line, emitted once per jittered run, is the authoritative record:

```
[Parallel] JIT NOP pool: 64 sleds, max N NOPs,
           mixed, backend: MAP_JIT+RWX

or

[Parallel] JIT NOP pool: 64 sleds, max N NOPs,
           pure (static), backend: static precompiled
```

Listing 5: Authoritative runtime backend record

The current `jit_nop_available()` implementation returns true because the static backend is always available. Therefore the UI availability state does not independently prove that `MAP_JIT` succeeded. It proves only that some pool backend is available. The backend log in Listing 5 must be retained for every run.

On the test device the realized backend was resolved directly. Under the attached debugger used for this test, execution was observed inside `try_static_backend()`, which the factory in Listing 4 reaches only after both `MAP_JIT` mappings have returned `MAP_FAILED`. This establishes that both `MAP_JIT` mapping attempts failed in the tested iPhone 17e / iOS 26.5.1 environment. The realized backend for the JIT-jitter family was therefore the static, pre-compiled pool, whose emitted instructions are pure NOP regardless of the requested mix.

2.6 Lock-free handoff and exact syscall insertion points

The concurrency mechanism is a per-worker-pair SPSC ring using acquire/release publication. The ring keeps allocation and release on separate worker threads and prevents a local-free fallback from collapsing both lifetime transitions into one worker thread's execution path. Timing jitter is then executed immediately before the userland allocation or release syscall, not inside the kernel helper.

Listing 6: Per-pair SPSC ring publication

```
static bool ring_push_item(port_item_ring_t *r, port_item_t item)
```

```
{
    uint64_t head = atomic_load_explicit(&r->head, memory_order_relaxed);
    uint64_t tail = atomic_load_explicit(&r->tail, memory_order_acquire);
    if (head - tail >= RING_SIZE_FLOOD) return false;
    r->slots[head & RING_MASK_FLOOD] = item;
    atomic_store_explicit(&r->head, head + 1, memory_order_release);
    return true;
}

static bool ring_pop_item(port_item_ring_t *r, port_item_t *out)
{
    uint64_t tail = atomic_load_explicit(&r->tail, memory_order_relaxed);
    uint64_t head = atomic_load_explicit(&r->head, memory_order_acquire);
    if (tail == head) return false;
    *out = r->slots[tail & RING_MASK_FLOOD];
    atomic_store_explicit(&r->tail, tail + 1, memory_order_release);
    return true;
}
```

```
// Allocation worker: timing sled -> syscall -> cross-thread handoff
if (g_jit_pool) jit_nop_execute_random(g_jit_pool);
else if (nm > 0) nop_sled(iter % nm);
kr = mach_port_allocate(task, MACH_PORT_RIGHT_RECEIVE, &port);
while (!ring_push_item(ctx->item_ring, (port_item_t){ .port = port }))
    __asm__ volatile("yield" ::: "memory");

// Free worker: receive handoff -> timing sled -> release syscall
if (ring_pop_item(ctx->item_ring, &item)) {
    if (g_jit_pool) jit_nop_execute_random(g_jit_pool);
    else if (nm > 0) nop_sled(iter % nm);
    kr = mach_port_mod_refs(task, item.port,
        MACH_PORT_RIGHT_RECEIVE, -1);
}
```

Listing 7: Jittered FLOOD allocation and release syscall boundaries

FAST and PIPELINE use the same jitter selection before each `mach_port_allocate()`. FAST also jitters each receive-right release. PIPELINE takes the send-right teardown branch through `zone_phase_destroy_ports()`, which does not contain a sled call; its message send and release side therefore remain unjittered in this revision.

2.7 Mode-specific syscall coverage

The sled is executed immediately before selected userland syscalls. It never delays the helper directly and cannot place a pause between `IRG` and `STG`/`ST2G`/`STGM`. Its purpose is to decorrelate the relative arrival times of alloc and free workers at the kernel boundary.

Table 3: Actual NOP/JIT insertion coverage in the inspected source revision

Mode	Allocation side	Free side	Important asymmetry
FAST	Before each `mach_port_allocate()`.	Before each receive-right `mach_port_mod_refs(..., -1)`.	Both lifetime transitions are jittered.
PIPELINE	Before each `mach_port_allocate()`.	No sled in the send-right teardown branch.	`mach_msg()` and pipeline teardown are not jittered; only allocation entry is perturbed.
FLOOD	Before each one-port `mach_port_allocate()`.	Before each one-port receive-right release.	Symmetric per-item jitter.
`PREFILL_FLOOD`	Same FLOOD hot path after reserve setup.	Same FLOOD hot path; reserve churn itself has no JIT call in the shown worker.	Burst path is jittered; reserve setup/churn is a separate state perturbation.
`PENDING_FLOOD`	Same FLOOD hot path after queued-message setup.	Same FLOOD hot path.	Pending-message prefill sends are not jittered; the later FLOOD is.

3 Real-time scheduling execution family

The real-time switch requests `THREAD_TIME_CONSTRAINT_POLICY` with a 1 ms period, 500 microseconds of computation, a 1 ms constraint, and preemptible execution. The request is best-effort; the implementation records only whether at least one FLOOD worker obtained the policy [7, 4].

```
thread_time_constraint_policy_data_t policy = {
    .period      = 1 * ms_to_abs,
    .computation = 500 * (ms_to_abs / 1000),
    .constraint  = 1 * ms_to_abs,
    .preemptible = 1
};
thread_policy_set(mach_thread_self(),
```

```
    THREAD_TIME_CONSTRAINT_POLICY,
    (thread_policy_t)&policy,
    THREAD_TIME_CONSTRAINT_POLICY_COUNT);
```

Listing 8: Best-effort real-time policy requested by FLOOD workers

```
static void *zone_flood_alloc_worker(void *arg)
{
    worker_ctx_t *ctx = (worker_ctx_t *)arg;
    set_affinity_tag(ctx->affinity);
    if (try_realtime_priority())
        atomic_store(&g_rt_granted, true);
    wait_for_start();
    /* one-port allocation loop */
}
```

Listing 9: FLOOD-family workers request real-time policy before the start barrier

The free-side FLOOD worker repeats the same call sequence. Despite the UI switch being visible for every mode, `try_realtime_priority()` is called only by `zone_flood_alloc_worker()` and `zone_flood_free_worker()`. FAST and PIPELINE use different worker functions and never call the real-time request. PREFILL and PENDING reuse the FLOOD worker path, so the option applies to them.

Table 4: Effective scope of the real-time option

Mode	Policy request executed?	Interpretation of the RT-labelled run
FAST	No	Scheduling path is equivalent to baseline with respect to this switch.
PIPELINE	No	Scheduling path is equivalent to baseline with respect to this switch.
FLOOD	Yes	RT was requested; actual grant requires the run's `RT priority: granted/denied` line.
`PREFILL_FLOOD`	Yes	FLOOD workers request RT after reserve setup.
`PENDING_FLOOD`	Yes	FLOOD workers request RT after pending-message setup.

Fifteen executions were performed, but the inspected source implements at most thirteen distinct scheduling/-timing configurations in that matrix: the FAST+RT and PIPELINE+RT executions do not add a real-time policy request. They remain useful repetitions of the baseline modes, but they are not separate RT treatments.

4 Fifteen-execution design

The test campaign exercised each of the five allocator-pressure modes once under each of three requested configuration families [12]. The third family is recorded as RT disabled because it was described separately from the real-time family.

Table 5: Requested configuration families

Family	Requested controls	Purpose
A: Baseline	RT off; NOP jitter off.	Reference behavior for the five established pressure modes.
B: RT option	*Realtime priority (FLOOD)* on; NOP jitter off.	Test whether best-effort time-constraint scheduling changes the manifestation rate. Effective only for FLOOD-family modes.
C: JIT jitter	NOP jitter on; JIT/static sled pool on; pool size 64; *Mixed NOP-equivalents* requested; RT not enabled in the stated family.	Randomize syscall-entry timing without changing the kernel trigger surface. Realized mix depends on the runtime backend.

Table 6: One-pass result matrix: fifteen executions

Mode	A: Baseline	B: RT option	C: 64-sled mixed request
Fast Zone Churn	**No external fault match**	**No external fault match** RT path not invoked	**No external fault match**
Zone Pipeline	**No external fault match**	**No external fault match** RT path not invoked	**No external fault match**
Flood (cache pressure)	**No external fault match**	**No external fault match** RT requested	**No external fault match**
Prefill + Flood	**No external fault match**	**No external fault match** RT requested	**No external fault match**
Pending Msg + Flood	**No external fault match**	**No external fault match** RT requested	**No external fault match**

This is a structured one-pass coverage matrix, not a full factorial experiment. It does not include the combined *RT plus JIT jitter* condition, a pure-NOP pool control, multiple pool sizes, multiple jitter ceilings, or repeated trials per cell. Accordingly, the result supports a descriptive statement about these fifteen executions; it does not establish a statistical upper bound on a rare fault probability.

5 Fault triage and Console capture

The following exact terms were tested. The filter set includes synchronous and asynchronous tag-check wording, exception and panic indicators, and the architectural tag-fault status-register names supplied for the campaign.

```
"DFSC 0x11"
"synchronous tag check fault"
"asynchronous tag check fault"
"tag check fault"
"tag mismatch"
TAG_FAULT
EXC_BAD_ACCESS
"Data Abort"
panicString
"panic(cpu"
TFSR
TFSRE0
```

Listing 10: Exact fault-filter terms used for the fifteen executions

A positive match must originate from `kernel`, a crash reporter, or a panic report and must contain fault context. Text printed by `MTETrigger.app` as search guidance is not fault evidence. In the available capture, `DFSC 0x11`, `synchronous tag check fault`, and `tag check fault` each matched the same fifteen guidance rows from the `MTETrigger` process. The other listed terms produced zero matches. After excluding the application's own messages, the external match count was zero [5].

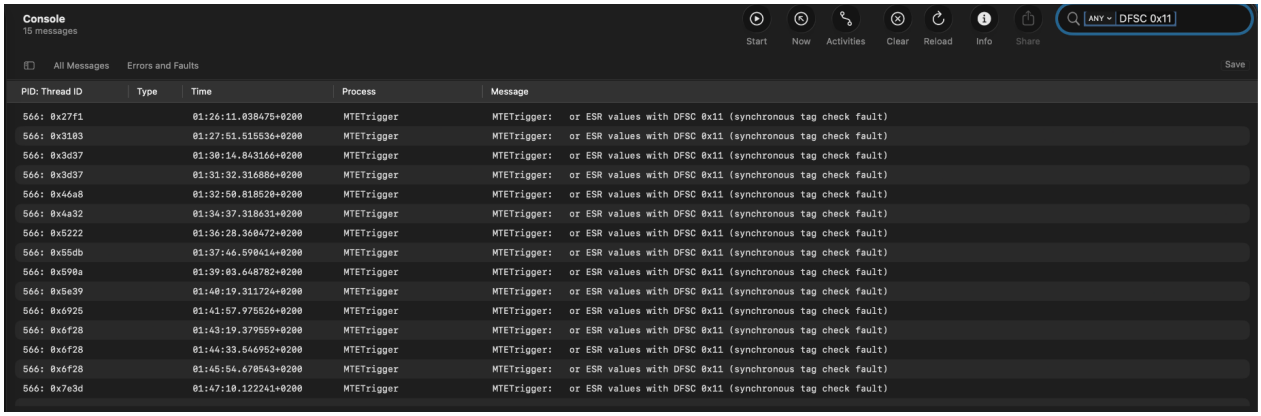


Figure 1: Representative Console result for `DFSC 0x11`. The filter returns fifteen rows, but every row is emitted by `MTETrigger` itself as part of the application's fault-search guide. No kernel, crash-reporter, or panic-report row is present.

```
DFSC 0x11:                    15 total, 15 MTETrigger, 0 external
synchronous tag check fault: 15 total, 15 MTETrigger, 0 external
tag check fault:             15 total, 15 MTETrigger, 0 external
asynchronous tag check fault: 0
tag mismatch:                 0
TAG_FAULT:                    0
EXC_BAD_ACCESS:               0
Data Abort:                   0
```

```
panicString / panic(cpu:      0
TFSR / TFSRE0:                0
```

Listing 11: Triage summary derived from the captured Console text

The correct dynamic classification is therefore *negative for an observed external MTE-fault manifestation.* It is not evidence that `irg_mte_function` was not executed, because successful tag generation and storage normally produce no public Console message.

6 Results and interpretation

The new campaign extends configuration coverage in two directions: scheduler policy and syscall-entry timing. No tested family produced a visible tag-check exception, an MTE-derived `EXC_BAD_ACCESS`, a data abort, or a panic signature. Within this single pass, neither the FLOOD-family RT request nor the realized sled-pool timing perturbation produced a black-box fault manifestation.

It should be emphasised that on the test device the requested mixed pool was not realized as mixed emission: as established in Section 2.5, both `MAP_JIT` mapping attempts failed in the tested iPhone 17e / iOS 26.5.1 environment under the attached debugger, so the JIT-jitter family executed the static, pure-NOP backend. The campaign therefore measured static timing perturbation, not runtime mixed-instruction emission.

Table 7: Evidence-supported conclusions from this addition

Claim	Status
The application exposes five pressure modes and the new jitter controls.	Supported by the SwiftUI and C source revision.
Random callable timing sleds are executed before selected Mach syscalls.	Supported by worker call sites in `mte_trigger.c`.
The mixed instruction set is available in the runtime JIT emitter.	Supported by `jit_nop.c`.
Every requested mixed run necessarily executed mixed instructions.	Not supported; on the test device the realized backend was static (pure NOP), so no requested mixed run emitted mixed instructions.
The requested `MAP_JIT` backend was realized on the test device.	Not supported; a debugger-attached observation reached `try_static_backend()`, establishing that both `MAP_JIT` mappings failed and the static fallback ran.
The real-time switch affected FAST and PIPELINE.	Not supported; those worker functions do not call the policy request.
Fifteen executions were represented in the Console capture.	Supported by fifteen occurrences of the application's fault-guide row and the author-supplied run order.
A real MTE tag fault occurred.	Not supported; external matches for the stated fault set were zero.
The same-object stale-tag race is absent.	Not established by a one-pass black-box negative result.

No per-run performance counters were supplied with the fifteen-run record, so this report does not compare throughput, ring stalls, overlap counts, allocation failures, thermal state, or port-name reuse proxies across families. Such comparisons would be necessary to determine whether RT or jitter increased useful kernel-path exposure or merely reduced transition rate.

The most important implementation distinction is *requested configuration versus realized configuration.* A reproducible run record must include the selected mode and UI controls, the `JIT NOP pool` backend line, the realized pure/mixed label, `max_nops`, pool size, `RT priority: granted/denied`, duration, worker count, alloc/free counters, and the exact Console time window. For the present campaign this distinction is now resolved on the timing axis: the realized backend was static pure NOP on every run, because the `MAP_JIT` backend was unavailable in the tested environment. The mixed-`MAP_JIT` condition therefore remains untested on iPhone 17e / iOS 26.5.1 and would require either a device configuration that grants `MAP_JIT` or an equivalent

JIT-enablement path. Without the remaining per-run fields, a negative result stays valid for fault manifestation, but differential claims between configurations remain weak.

7 Reproducibility-critical follow-up

Future black-box repetitions on the same jailed device should preserve the following minimum metadata. This does not require a breakpoint or kernel debugger; it improves the scientific value of the observable layer.

Table 8: Minimum per-run record for repeat testing

Field	Reason
Run ID, mode, start/end timestamp	Correlates app output, Console rows, crash reports, and unexpected reboots.
RT requested and RT granted/denied	Distinguishes UI intent from actual thread-policy admission.
NOP ceiling, pool size, requested mix	Defines the timing treatment.
Runtime backend and realized mix	Distinguishes `MAP_JIT` mixed emission from pure static fallback.
Allocation/free counts and overlaps	Normalizes exposure by completed lifetime transitions rather than wall-clock duration alone.
Failures, ring stalls, prefill/pending counters	Detects runs dominated by setup errors or userland backpressure.
Foreground/unlocked state and thermal observations	Identifies suspension and thermal throttling as confounders.
Console filter excluding process `MTETrigger`	Prevents the application's own guidance from being counted as a fault.
Post-reboot panic report check	Captures failures that cannot remain visible in the live Console stream.

For experimental completeness, the next matrix should add a combined RT-plus-JIT family, a pure-NOP pool control, and repeated trials in randomized order. Repetition is especially important because one execution per cell cannot characterize a rare race. Results should be normalized by successful port lifetime transitions and reported together with the runtime backend and RT grant state.

8 Conclusion

The new implementation materially broadens the black-box timing search space. It can perturb syscall entry with deterministic loop NOPs, random `MAP_JIT` sleds, or random signed static sleds; it can request mixed ARM64 timing instructions when the JIT backend is realized; and it can request time-constraint scheduling for FLOOD-family workers. The fifteen-execution campaign covered all five allocator-pressure modes under baseline, RT-labelled, and JIT-jitter-labelled configurations. On the iPhone 17e / iOS 26.5.1 test device the JIT-jitter family was realized as the static, pure-NOP backend, because both `MAP_JIT` mapping attempts failed in the tested debugger-attached environment; the mixed-instruction emission path was therefore requested but not exercised. No external MTE tag-fault signature was observed under the expanded filter set.

The codebase now contains the dynamic/userland methods described across Reports 01–04: the correlated zone and Skywalk triggers, zone-only parallel pressure, all five allocator-pressure modes, prefill and pending-message state shaping, optional real-time scheduling, background execution, overlap/statistics collection, and the new loop/JIT/static timing layer. Report 00 remains the static basis rather than an executable feature.

The result is negative for fault manifestation, not negative for helper execution or for the underlying ordering hypothesis. Source review also narrows the interpretation: FAST and PIPELINE do not enter the RT policy path, PIPELINE jitter is allocation-side only, and static fallback converts a requested mixed configuration into pure NOP sleds. Retaining the per-run backend line, RT grant state, counters, and repeated randomized trials is required before comparing the effectiveness of the timing treatments.

References

[1] Kaya Ercihan. *Investigating a Potential TOCTOU in the iOS 26.0.1 MTE Tagging Path*. Report 00. Independent Security Research, Mar. 1, 2026.

[2] Kaya Ercihan. *iOS 26.5 MTE Tagging Paths: Caller–Trigger Correlation and TOCTOU Preconditions*. Report 01. Independent Security Research, June 11, 2026.

[3] Kaya Ercihan. *Physical-Device Evaluation of a Potential TOCTOU in the iOS MTE Tagging Path on iPhone 17e / iOS 26.5.1*. Report 02. Independent Security Research, June 16, 2026.

[4] Kaya Ercihan. *Dynamic Pressure Validation for Potential iOS Kernel MTE Allocator-State Races*. Report 03. Independent Security Research, June 24, 2026.

[5] Kaya Ercihan. *Console filter capture*. Console capture spanning the fifteen executions, DFSC 0x11 produced fifteen matches, all from MTETrigger's own filter guide, with no external tag-fault match. Aug. 3, 2026.

[6] Kaya Ercihan. *MTETrigger SwiftUI Configuration Source Revision Used for Report 04*. Independent research artifact. Files include TriggerView.swift, TriggerViewModel.swift, and MTETriggerApp.swift; configuration and C-API propagation for trigger mode, real-time priority, NOP jitter, pool size, and instruction mix. Aug. 3, 2026.

[7] Kaya Ercihan. *MTETrigger C Source Revision Used for Report 04*. Independent research artifact. Files include mte_trigger.c and mte_trigger.h; contains timing-jitter controls, worker insertion points, real-time policy requests, counters, and runtime backend logging. Aug. 3, 2026.

[8] Kaya Ercihan. *MTETrigger Timing-Sled Source Revision Used for Report 04*. Independent research artifact. Files include jit_nop.c and jit_nop.h; runtime-generated and statically precompiled ARM64 timing-sled pool implementation. Aug. 3, 2026.

[9] Arm Limited. *Arm Architecture Reference Manual for A-profile Architecture*. Document DDI 0487, Issue M.a; architectural reference for MTE, hint/event instructions, thread-visible effects, and memory-ordering barriers. 2026. URL: `https://developer.arm.com/documentation/ddi0487/maa`.

[10] Kaya Ercihan. *MTETrigger Development Entitlements Used for Report 04*. Independent research artifact. Development entitlement set requesting get-task-allow, disabled library validation, and allow-jit. Aug. 3, 2026.

[11] Apple Inc. *Allow Execution of JIT-Compiled Code Entitlement*. Apple Developer Documentation; describes the Hardened Runtime entitlement associated with writable/executable memory using the MAP_JIT flag. URL: `https://developer.apple.com/documentation/BundleResources/Entitlements/com.apple.security.cs.allow-jit` (visited on 10/03/2026).

[12] Kaya Ercihan. "Fifteen-run MTETrigger timing-perturbation test matrix". Author-supplied execution record: five allocator-pressure modes under baseline, real-time-requested, and JIT-sled timing-jitter configurations. Aug. 3, 2026.